Sistema de Cadastro de Produtos

MVP em Microsservicos na Microsoft Azure

DOCUMENTO DE ARQUITETURA

TechStore Cloud | Pos-graduacao em Engenharia de Software - InfNet

Versao 1.0 | 05/10/2026

1. Contexto e objetivo

A TechStore Cloud (empresa ficticia) lancara uma nova linha de produtos digitais e precisa de um sistema moderno e escalavel para o cadastro desses produtos. Este documento descreve a arquitetura de um **MVP** do Sistema de Cadastro de Produtos, organizado em microsservicos e hospedado na Microsoft Azure, seguindo boas praticas de engenharia de software: separacao de responsabilidades, infraestrutura como codigo, seguranca por padrao e observabilidade desde o primeiro dia.

1.1 Requisitos atendidos

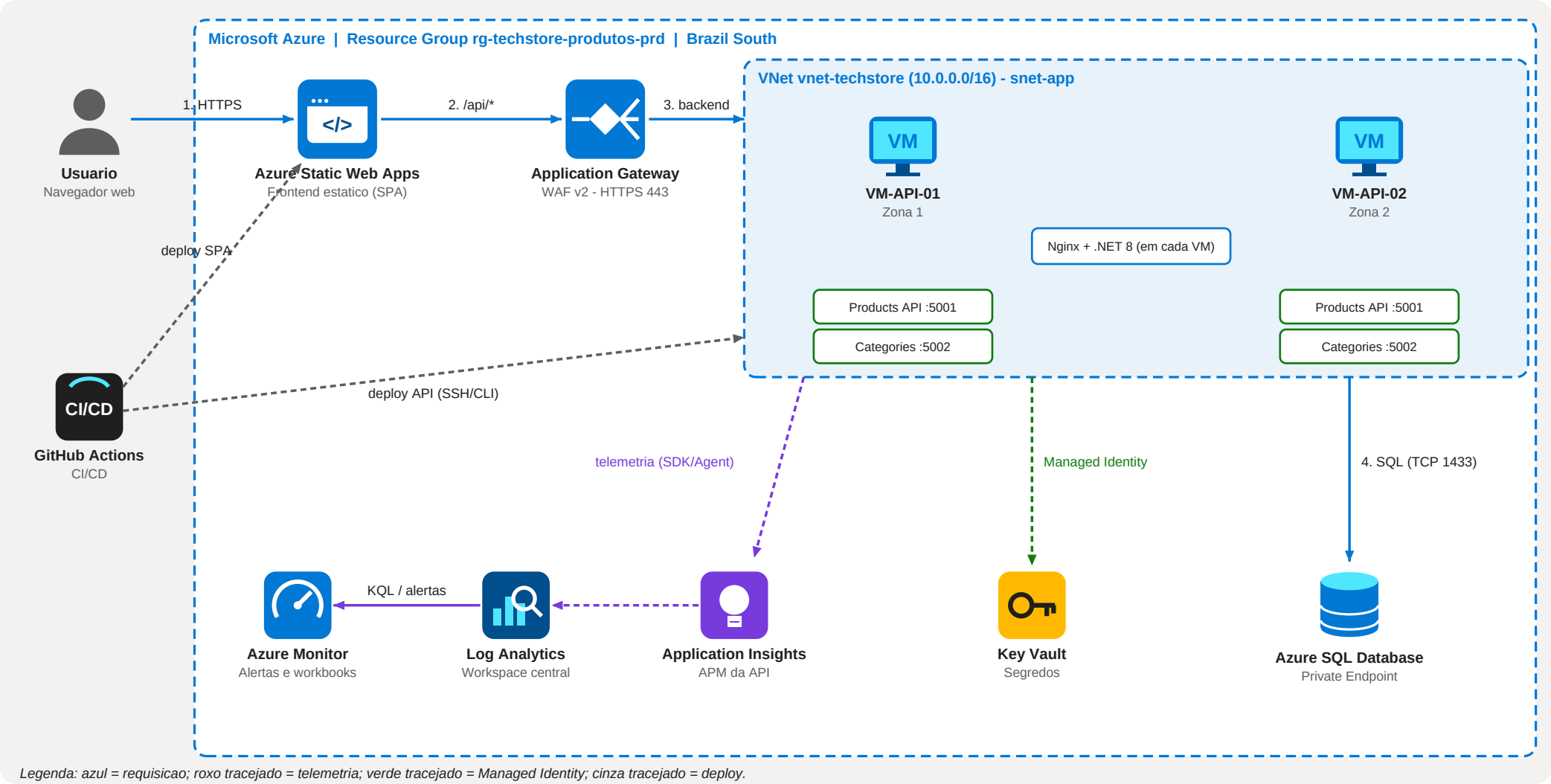
Requisito do projeto	Decisao arquitetural
Frontend estatico	**Azure Static Web Apps** (SPA em HTML/JS/CSS), com CDN global, TLS gerenciado e deploy via GitHub Actions. Alternativa equivalente: Static website no Azure Blob Storage + Azure CDN/Front Door.
API REST em .NET 6+	**ASP.NET Core 8 (LTS)** Web API, um servico por dominio (Products e Categories), com OpenAPI/Swagger, health checks e autenticacao JWT.
Banco de dados	**Azure SQL Database** (PaaS), acessado exclusivamente por Private Endpoint; um schema por microsservico.
Hospedagem da API em VMs	**Azure Virtual Machines** Linux (Ubuntu LTS) em duas Availability Zones, atras de um **Application Gateway WAF v2**.
Monitoramento	**Azure Monitor + Log Analytics** como plataforma central, com **Application Insights** (workspace-based) para APM da API.

1.2 Requisitos nao funcionais

- **Disponibilidade:** meta de 99,9% no MVP, com duas VMs em zonas distintas e health probes no gateway.
- **Escalabilidade:** API stateless (escala horizontal adicionando VMs ou usando VM Scale Set); frontend servido por CDN; banco com possibilidade de aumentar vCores sem mudanca de codigo.
- **Seguranca:** trafego cifrado (TLS 1.2+), WAF, NSGs, Managed Identity, segredos no Key Vault e banco sem exposicao publica.
- **Operabilidade:** logs, metricas e traces correlacionados, alertas proativos e deploy automatizado.

2. Visao geral da arquitetura

A figura abaixo apresenta a visao logica de ponta a ponta. O usuario acessa a SPA hospedada no Static Web Apps; o navegador chama a API REST publicada atras do Application Gateway, que distribui as requisicoes entre as VMs que executam os microsservicos; os servicos persistem dados no Azure SQL Database. Toda a telemetria converge para o Log Analytics, e o Azure Monitor oferece dashboards e alertas.

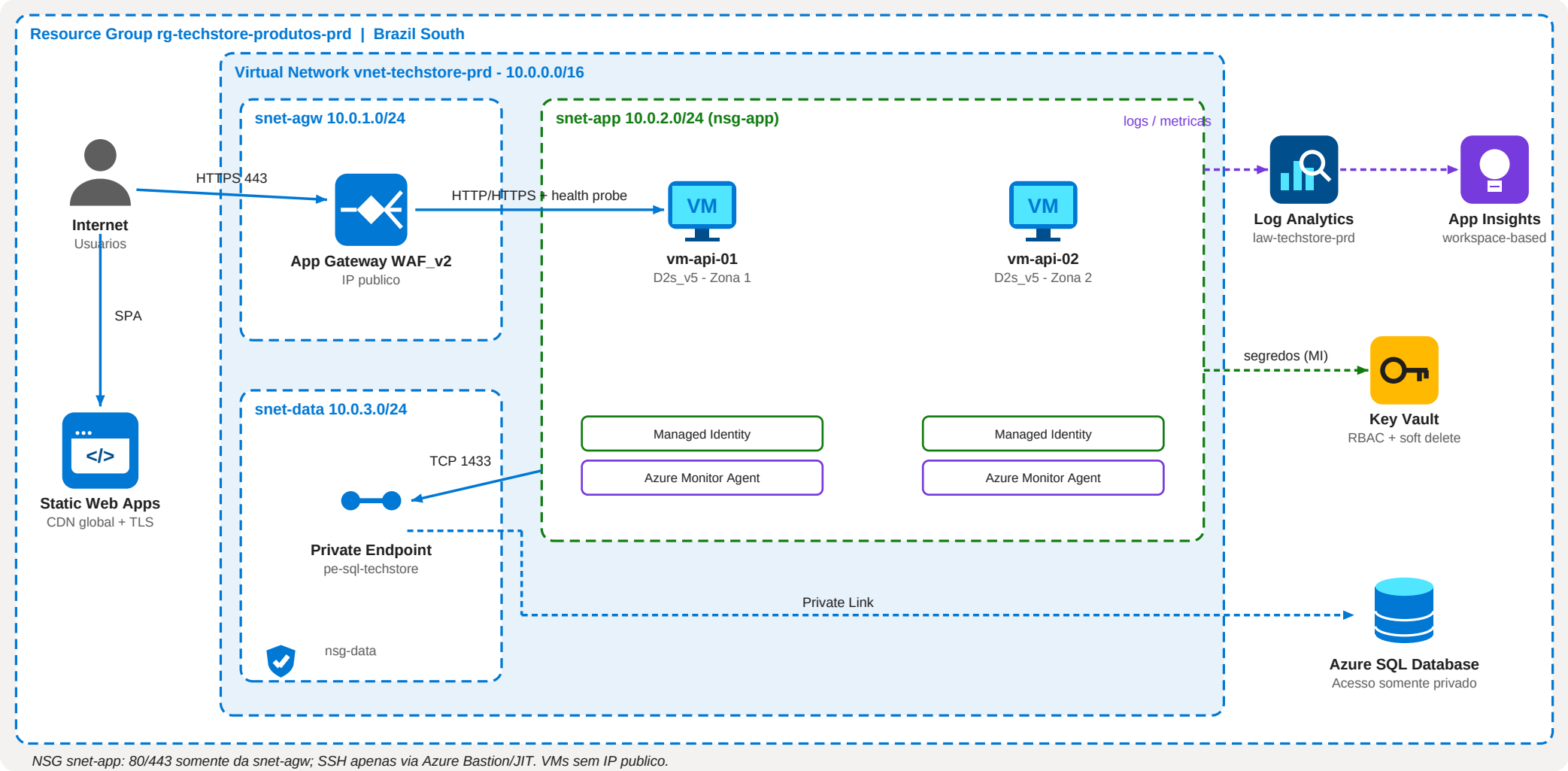


Legenda: azul = requisicao; roxo tracejado = telemetria; verde tracejado = Managed Identity; cinza tracejado = deploy.

Figura 1 - Visao logica da solucao (icones no padrao visual Azure).

3. Arquitetura de infraestrutura e rede

A API e implantada em uma Virtual Network dedicada, com tres sub-redes e isolamento por NSG. Somente o Application Gateway possui IP publico; as VMs nao sao expostas diretamente. O Azure SQL e acessado por Private Endpoint na sub-rede de dados, e os segredos sao lidos do Key Vault via Managed Identity.



NSG snet-app: 80/443 somente da snet-agw; SSH apenas via Azure Bastion/JIT. VMs sem IP publico.

Figura 2 - Topologia de implantacao: VNet, sub-redes, NSGs e servicos PaaS.

4. Fluxo de cadastro de produto

A sequencia abaixo detalha o caso de uso principal do MVP (cadastrar produto), incluindo validacao, persistencia e emissao de telemetria correlacionada.

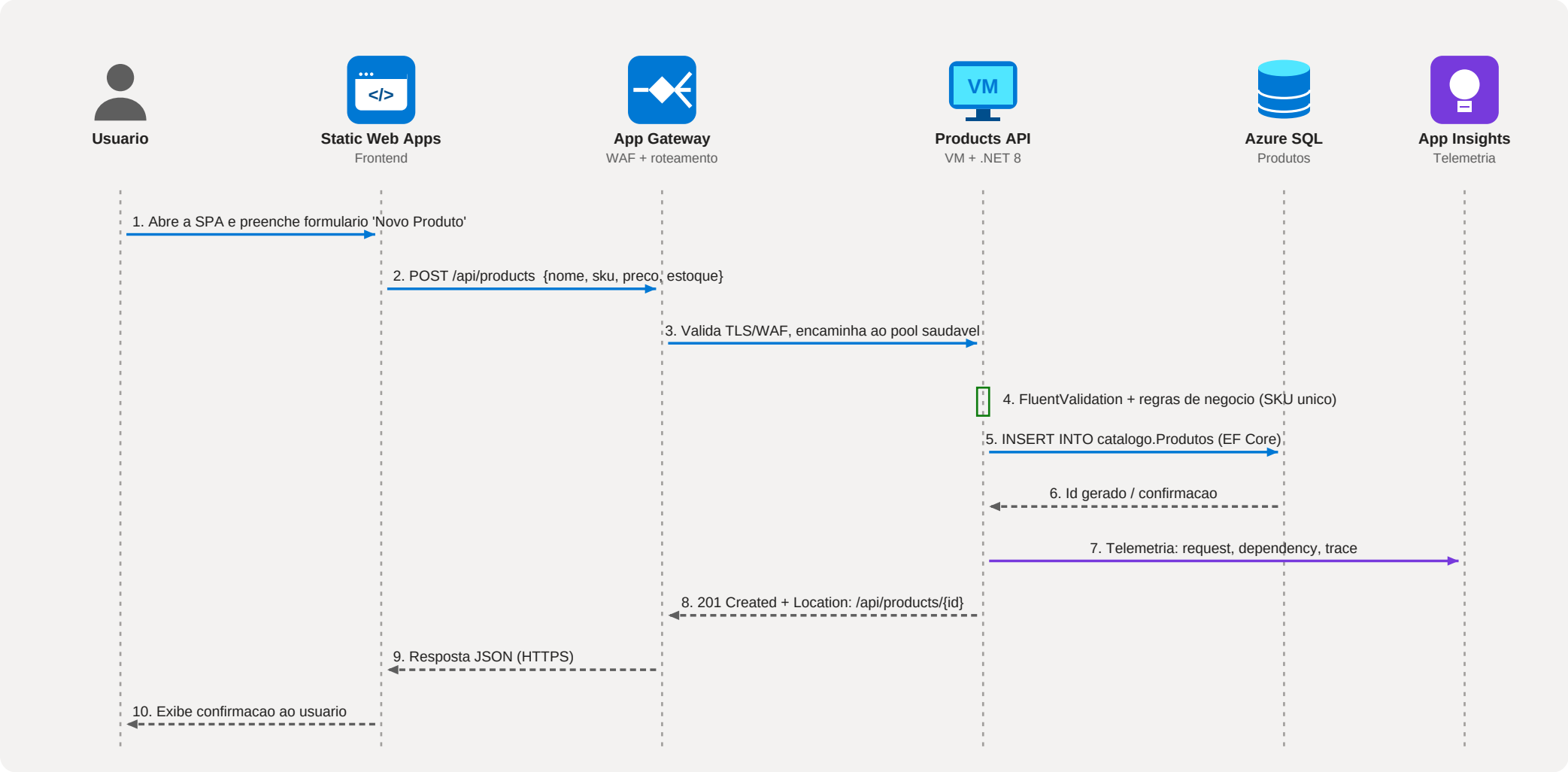


Figura 3 - Diagrama de sequencia do cadastro de produto.

5. Microsservicos e API REST

O MVP adota dois microsservicos independentes, cada um com seu projeto, pipeline, schema de banco e rota no gateway. As VMs executam os servicos como unidades *systemd* (ou containers Docker) atras de um proxy reverso Nginx. Essa separacao permite migrar futuramente para AKS ou Container Apps sem reescrever o dominio.

Servico	Porta / rota	Responsabilidade	Schema SQL
Products API	5001 | /api/products/*	CRUD de produtos digitais (nome, SKU, descricao, preco, estoque/licencas, status)	catalogo
Categories API	5002 | /api/categories/*	CRUD de categorias e associacao com produtos	categorias

5.1 Endpoints principais (Products API)

Metodo	Rota	Descricao	Retorno
POST	/api/products	Cadastra produto	201 Created + Location
GET	/api/products?page=1&size=20&q=	Lista paginada com busca	200 OK
GET	/api/products/{id}	Consulta por Id	200 / 404
PUT	/api/products/{id}	Atualiza produto	204 / 404 / 409
DELETE	/api/products/{id}	Inativa produto (soft delete)	204 / 404
GET	/health | /health/ready	Liveness e readiness (inclui SQL)	200 / 503

5.2 Modelo de dados (resumo)

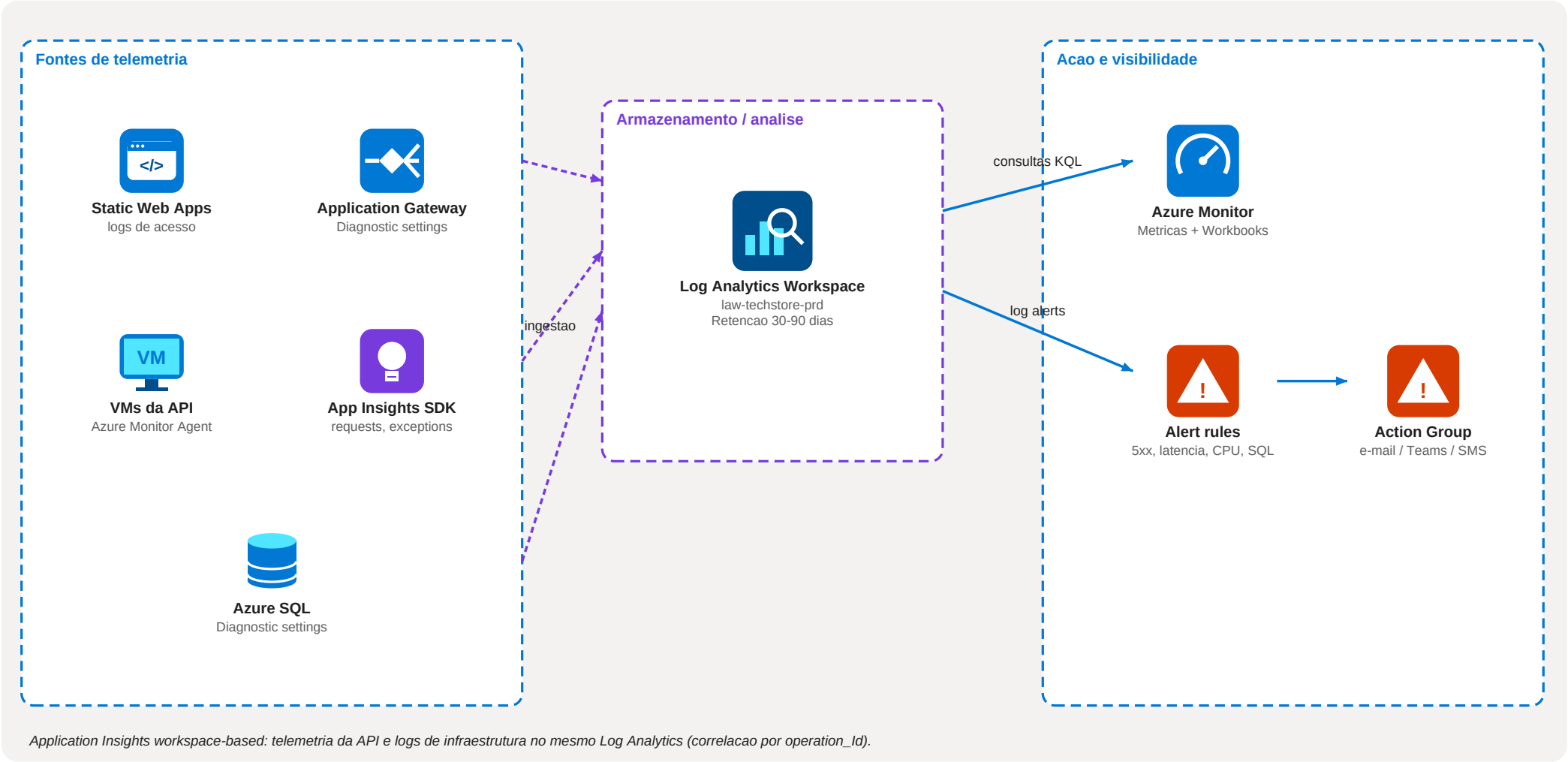
```
catalogo.Produtos ( Id UNIQUEIDENTIFIER PK, Sku NVARCHAR(40) UNIQUE, Nome NVARCHAR(150), Descricao NVARCHAR(MAX),
                    Preco DECIMAL(12,2), Moeda CHAR(3), Estoque INT, CategoriaId UNIQUEIDENTIFIER,
                    Ativo BIT, CriadoEm DATETIME2, AtualizadoEm DATETIME2, RowVersion ROWVERSION )
categorias.Categorias ( Id UNIQUEIDENTIFIER PK, Nome NVARCHAR(80) UNIQUE, Descricao NVARCHAR(250) )
```

5.3 Boas praticas adotadas

- Arquitetura em camadas (API, Application, Domain, Infrastructure) e injecao de dependencias nativa do .NET.
- EF Core com migrations, politica de retry para falhas transitorias e controle de concorrencia otimista (RowVersion).
- Validacao com FluentValidation, respostas de erro no padrao *ProblemDetails* (RFC 7807) e versionamento de API (/api/v1).
- CORS restrito ao dominio do Static Web Apps; logs estruturados (Serilog/ILogger) enviados ao Application Insights.

6. Observabilidade e monitoramento

O Azure Monitor e a camada unificadora: metricas de plataforma, logs de diagnostico (Application Gateway, Azure SQL, Static Web Apps), telemetria de aplicacao (Application Insights) e logs de sistema das VMs (Azure Monitor Agent com Data Collection Rule) sao armazenados no mesmo Log Analytics Workspace.



Application Insights workspace-based: telemetria da API e logs de infraestrutura no mesmo Log Analytics (correlacao por operation_Id).

Figura 4 - Pipeline de monitoramento, consulta e alertas.

6.1 Alertas recomendados

Alerta	Fonte	Condicao	Severidade
Taxa de erros 5xx da API	Application Insights	> 2% em 5 min	Sev 1
Latencia p95	Application Insights	> 1 s em 10 min	Sev 2
Backend do gateway nao saudavel	Application Gateway	UnhealthyHostCount > 0	Sev 1
CPU / memoria das VMs	Azure Monitor Agent	CPU > 80% por 10 min	Sev 2
Uso de CPU / armazenamento do SQL	Azure SQL	> 80% por 15 min	Sev 2
Falhas de login / WAF bloqueios	Log Analytics (KQL)	Pico anomalo	Sev 3

6.2 Exemplo de consulta KQL (requisicoes lentas nas ultimas 24 h)

```
AppRequests
| where TimeGenerated > ago(24h) and Name startswith "POST /api/products"
| summarize p95 = percentile(DurationMs, 95), falhas = countif(Success == false) by bin(TimeGenerated, 15m)
| order by TimeGenerated desc
```

7. Seguranca

- **Borda:** Application Gateway WAF v2 (regras OWASP) com TLS 1.2+; Static Web Apps com certificado gerenciado.
- **Rede:** VMs sem IP publico; NSGs restringem 80/443 a sub-rede do gateway; acesso administrativo apenas via Azure Bastion ou JIT.
- **Identidade:** Managed Identity nas VMs para acessar Key Vault e Azure SQL (Entra ID auth), sem senhas no codigo; usuarios da aplicacao autenticados com JWT (Microsoft Entra ID).
- **Dados:** Azure SQL com firewall publico desabilitado, Private Endpoint, TDE e auditoria enviada ao Log Analytics; backups automaticos (PITR).
- **Segredos e governanca:** Key Vault com RBAC e soft delete; Azure Policy e tags (ambiente, centro de custo); Microsoft Defender for Cloud habilitado.

8. Escalabilidade, resiliencia e custos

- **Escala horizontal da API:** VMs identicas criadas a partir de imagem/cloud-init; evolucao natural para Virtual Machine Scale Set com autoscale por CPU e requisicoes.
- **Resiliencia:** duas VMs em Availability Zones diferentes, health probes, reinicio automatico pelo systemd, retries com backoff (Polly) e circuit breaker nas chamadas ao banco.
- **Banco:** Azure SQL com backups automaticos e restauracao pontual; replica geo (opcional) para DR em fase posterior.
- **Custos (MVP):** usar SKUs menores em desenvolvimento/homologacao (B-series, Static Web Apps Free, SQL Basic/Serverless), desligar VMs de nao-producao fora do horario e acompanhar via Azure Cost Management. Valores devem ser estimados na Azure Pricing Calculator para a regiao escolhida.

9. CI/CD e infraestrutura como codigo

- **Repositorio:** monorepo (ou repos por servico) no GitHub; branch protegida com revisao obrigatoria.
- **Frontend:** GitHub Actions publica automaticamente no Static Web Apps a cada merge em main (preview por pull request).
- **API:** pipeline restaura, testa (xUnit), publica o artefato (dotnet publish) e implanta nas VMs via Azure CLI/SSH com troca gradual entre VM-01 e VM-02 (rolling).
- **Infraestrutura:** Bicep (ou Terraform) para recursos: VNet, NSGs, App Gateway, VMs, SQL, Key Vault, Log Analytics e Application Insights.

10. Riscos e evolucao

Risco / limitacao do MVP	Mitigacao / evolucao
Gerenciar patches e SO das VMs aumenta o esforco operacional	Atualizacoes automaticas (Azure Update Manager), imagens padronizadas; migrar para Container Apps/AKS
Banco compartilhado entre servicos (acoplamento)	Schemas separados e acesso por usuario dedicado; evoluir para bancos por servico
Comunicacao entre servicos apenas sincrona	Introduzir Azure Service Bus para eventos (ProdutoCriado, ProdutoAtualizado)
Ponto unico de entrada (gateway)	Gateway zonal redundante ja previsto; Azure Front Door para multi-regiao

11. Observacao sobre os icones

Os diagramas utilizam icones vetoriais desenhados no padrao visual dos servicos Azure (cores, formas e nomenclatura). Para a entrega final, recomenda-se substituir pelos icones oficiais do pacote *Azure Architecture Icons* da Microsoft (disponivel em learn.microsoft.com/azure/architecture/icons), mantendo a mesma disposicao.